SYMFONY 7

FRAMEWORK PHP

SYMFONY - SOMMAIRE

- Introduction à Symfony
- Architecture de Symfony
- Installation & configuration
- Les bases de Symfony
- Gestion des données
- Sécurité & authentification
- Tests & déploiement

INTRODUCTION

FRAMEWORK SYMFONY

INTRODUCTION

- Symfony est un framework PHP open-source utilisé pour le développement d'applications web.
- Il suit l'architecture MVC et fournit des outils puissants pour accélérer le développement.

QU'EST-CE QUE SYMFONY ?

- Symfony est un ensemble de composants PHP réutilisables et un framework pour créer des applications web robustes.
- Créé par SensioLabs en 2005, il est utilisé par de nombreuses entreprises et projets open-source.

POURQUOI UTILISER SYMFONY ?

- Modularité et flexibilité : Composants réutilisables
- Performances optimisées
- Sécurité avancée intégrée
- Documentation complète et communauté active

COMPARAISON AVEC D'AUTRES FRAMEWORKS PHP

- Symfony vs Laravel :
- Symfony est plus flexible mais plus complexe
- Laravel est plus accessible pour les débutants

COMPARAISON AVEC D'AUTRES FRAMEWORKS PHP

- Symfony vs CodeIgniter :
- Symfony offre une meilleure architecture et plus de fonctionnalités avancées

COMPARAISON AVEC D'AUTRES FRAMEWORKS PHP

- Symfony vs Zend Framework :
- Symfony est plus populaire et bénéficie d'un support communautaire plus large

LES CAS D'UTILISATION DE SYMFONY

- Développement d'applications web complexes
- API et microservices
- Plateformes e-commerce
- Applications d'entreprise

LES GRANDES ENTREPRISES QUI UTILISENT SYMFONY

- Spotify
- BlaBlaCar
- Dailymotion
- Drupal (CMS)

ARCHITECTURE

FRAMEWORK SYMFONY

ARCHITECTURE DE SYMFONY

- Symfony est basé sur une architecture modulaire et suit le modèle MVC (Modèle-Vue-Contrôleur).

LE MODÈLE MVC DANS SYMFONY

- **Modèle (Model)** : Gère les données et la logique métier.
- **Vue (View)** : Gère l'affichage des informations.
- **Contrôleur (Controller)** : Gère les requêtes et les interactions entre le modèle et la vue.

LES COMPOSANTS DE SYMFONY

- Symfony est constitué de nombreux composants réutilisables :
- **HttpFoundation** : Gestion des requêtes et réponses HTTP
- **Routing** : Système de routage avancé
- **Security** : Gestion des utilisateurs et de l’authentification
- **Doctrine ORM** : Interaction avec la base de données
- **Twig** : Moteur de templates

LE CYCLE DE VIE D'UNE REQUÊTE DANS SYMFONY

1. Un utilisateur envoie une requête HTTP.
2. Le routeur identifie le contrôleur approprié.
3. Le contrôleur traite la requête et interagit avec le modèle.
4. Le modèle récupère ou met à jour les données.
5. Le contrôleur envoie les données à la vue.
6. La vue génère une réponse HTTP envoyée à l'utilisateur.

LE SYSTÈME DE ROUTAGE

- Permet de définir des URL pour chaque action de l'application.
- Utilisation d'annotations, de YAML ou de fichiers PHP pour définir les routes.
- Gestion avancée des paramètres et des contraintes d’URL.

LE MOTEUR DE TEMPLATES TWIG

- Syntaxe simplifiée pour gérer l'affichage des données.
- Séparation de la logique métier et de l'affichage.
- Sécurisation des sorties pour éviter les injections XSS.

DOCTRINE ORM ET GESTION DE LA BASE DE DONNÉES

- Utilisation d'objets pour interagir avec la base de données.
- Mapping Objet-Relationnel (ORM) via Doctrine.
- Système de migrations pour gérer l'évolution du schéma de la base de données.

LE COMPOSANT SECURITY

- Gestion des utilisateurs et de l’authentification.
- Système de rôles et permissions.
- Protection contre les attaques CSRF et SQL Injection.

SYMFONY FLEX ET SON IMPACT SUR L'ARCHITECTURE

- Système de gestion des dépendances optimisé.
- Installation simplifiée des composants nécessaires.
- Flexibilité pour créer des applications légères ou complexes.

INSTALLATION ET CONFIGURATION

FRAMEWORK SYMFONY

PRÉREQUIS

- PHP 8.1 ou supérieur
- Composer
- Serveur Web
- Base de données

INSTALLATION VIA COMPOSER

1. Installer Composer : https://getcomposer.org
2. Installer Symfony CLI :
   - https://symfony.com/download
3. Vérifier l'installation :
   - symfony check:req

CRÉER UN PROJET SYMFONY

- Commande pour créer un projet :
- composer create-project symfony/skeleton mon_projet
- cd mon_projet
- symfony server:start

CONFIGURATION DE LA BASE DE DONNÉES

- Exemple de configuration dans `.env` :
  - DATABASE_URL="mysql://user:password@127.0.0.1:3306/nom_base"

LES BASES

FRAMEWORK SYMFONY

LES BASES DE SYMFONY

Symfony repose sur une architecture claire et des concepts fondamentaux permettant un développement structuré et efficace.

LES ROUTES DANS SYMFONY

Définition

```
use Symfony\Component\Routing\Attribute\Route;

#[Route('/book')]
class BookController extends AbstractController
{
    #[Route('/', name: 'app_book_index', methods: ['GET'])]
    public function index(BookRepository $bookRepository): Response
    {
        return $this->render('book/index.html.twig', [
            'books' => $bookRepository->findAll(),
        ]);
    }
```

LES CONTRÔLEURS

- Exemple de c

```php
<?php

namespace App\Controller;

use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;

class AccueilController extends AbstractController
{
    #[Route('/', name: 'homepage')]
    public function index(): Response
    {
        return new Response('<h1>Bienvenue sur Symfony !</h1>');
    }
}
?>
```

EXPLICATIONS

- Déclaration du namespace :

```
namespace App\Controller;
```

Indique que ce fichier fait partie du dossier src/Controller.

EXPLICATIONS

- Importation des classes nécessaires :

```
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
```

•AbstractController : Fournit des méthodes Symfony utiles (comme render() pour afficher des templates).
•Response : Permet de retourner une réponse HTTP.
•Route : Permet de définir une URL pour ce contrôleur.

EXPLICATIONS

- Déclaration de la classe AccueilController :

```
class AccueilController extends AbstractController
```

- Étend AbstractController, ce qui permet d'utiliser des fonctionnalités avancées.

EXPLICATIONS

- Définition de la route :

```
#[Route('/', name: 'homepage')]
```

•/ : URL associée à cette méthode.
•name: 'homepage' : Nom unique de la route (utile pour la redirection ou les liens).

EXPLICATIONS

- Méthode index() :

```
public function index(): Response
{
    return new Response('<h1>Bienvenue sur Symfony !</h1>');
}
```

- Retourne une réponse HTTP avec du contenu HTML.

CONTRÔLEUR AVEC UN TEMPLATE TWIG

```
#[Route('/', name: 'homepage')]
public function index(): Response
{
    return $this->render('accueil/index.html.twig', [
        'message' => 'Bienvenue sur Symfony !'
    ]);
}
```

CONTRÔLEUR AVEC UN TEMPLATE TWIG

- le **fichier Twig** correspondant (templates/accueil/index.html.twig) :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Accueil</title>
</head>
<body>
    <h1>{{ message }}</h1>
</body>
</html>
```

GESTION DES FORMULAIRES

- Exemple d'un formulaire avec Symfony :

```
use Symfony\Component\Form\Extension\Core\Type\TextType;
use Symfony\Component\Form\Extension\Core\Type\SubmitType;

$form = $this->createFormBuilder()
    ->add('nom', TextType::class)
    ->add('envoyer', SubmitType::class)
    ->getForm();
```

GESTION DES DONNÉES

FRAMEWORK SYMFONY

GESTION DES DONNÉES DANS SYMFONY

- Symfony utilise Doctrine ORM pour gérer la base de données. Il facilite la manipulation des données via des entités.

PRÉSENTATION DE DOCTRINE ORM

- Doctrine est un ORM (Object-Relational Mapping) pour PHP.
- Permet de gérer les bases de données en utilisant des objets PHP.
- Supporte MySQL, PostgreSQL, SQLite, etc.

INSTALLATION DE DOCTRINE

- Pour installer Doctrine dans un projet Symfony :

https://symfony.com/doc/current/doctrine.html

CONFIGURATION DE LA BASE DE DONNÉES

- Symfony utilise un fichier `.env` pour gérer la connexion à la base de données.
  - Exemple de configuration pour MySQL :

```
# DATABASE_URL="sqlite:///%kernel.project_dir%/var/data.db"
DATABASE_URL="mysql://root:password@localhost:3306/projet?serverVersion=8.0.30&charset=utf8mb4"
#DATABASE_URL="mysql://root:root@localhost:3306/biblio?serverVersion=11.2.2-MariaDB&charset=utf8mb4"
#DATABASE_URL="postgresql://root@127.0.0.1:5432/biblio?serverVersion=16&charset=utf8"
###< doctrine/doctrine-bundle ###
```

CRÉATION D'UNE ENTITÉ

- Une entité est une représentation d'une table en base de données.
- Création d'une entité `Utilisateur` :

```
php bin/console make:entity Utilisateur
```

DÉFINITION D'UNE ENTITÉ

- Exemple d'une classe

```php
namespace App\Entity;

use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Utilisateur
{
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column(type: 'integer')]
    private int $id;

    #[ORM\Column(type: 'string', length: 100)]
    private string $nom;
}
```

CRÉATION DES MIGRATIONS

- Après avoir défini l'entité, il faut générer la migration :

```
php bin/console make:migration
```

```
php bin/console doctrine:migrations:migrate
```

INTERAGIR AVEC LA BASE DE DONNÉES

- Utilisation du Repository pour récupérer des données :

```
$utilisateur = $this->getDoctrine()
    ->getRepository(Utilisateur::class)
    ->find(1);
```

AJOUT ET SUPPRESSION D'UN ENREGISTREMENT

- Exemple d'ajout d'un utilisateur :

```
$entityManager = $this->getDoctrine()->getManager();
$utilisateur = new Utilisateur();
$utilisateur->setNom('Jean Dupont');
$entityManager->persist($utilisateur);
$entityManager->flush();
```

AJOUT ET SUPPRESSION D'UN ENREGISTREMENT

- Suppression :

```
$entityManager->remove($utilisateur);
$entityManager->flush();
```

LES RELATIONS ENTRE ENTITÉS

- Doctrine permet de gérer les relations entre entités :
  1. **OneToOne** (Un utilisateur - un profil)
  2. **OneToMany** (Un utilisateur - plusieurs commandes)
  3. **ManyToMany** (Un utilisateur - plusieurs groupes)

EXEMPLE DE RELATION ONETOMANY

```php
<?php
class Utilisateur {
    /** @ORM\\OneToMany(targetEntity='Article', mappedBy='utilisateur') */
    private $articles;
}

class Article {
    /** @ORM\\ManyToOne(targetEntity='Utilisateur', inversedBy='articles') */
    private $utilisateur;
}
```

UTILISATION DU QUERYBUILDER

- Le QueryBuilder permet de construire dynamiquement des requêtes :

```
$qb = $this->createQueryBuilder('u')
->where('u.nom = :nom')
->setParameter('nom', 'Jean')
->getQuery()
->getResult();
```

Plus de détails sur: https://symfony.com/doc/current/doctrine.html

RÉSUMÉ DE LA GESTION DES DONNÉES

- Doctrine ORM facilite la gestion des bases de données.
- Définition des entités et relations.
- Création de migrations et interaction avec la base.
- Utilisation du QueryBuilder pour des requêtes avancées.

SÉCURITÉ & AUTHENTIFICATION

FRAMEWORK SYMFONY

SÉCURITÉ ET AUTHENTIFICATION DANS SYMFONY

Symfony fournit un système de sécurité puissant pour gérer l'authentification et l'autorisation des utilisateurs.

PRÉSENTATION DU COMPOSANT SECURITY

- Gestion des utilisateurs et de l'authentification.
- Gestion des rôles et permissions.
- Protection contre les attaques CSRF et injections SQL.

INSTALLATION DU COMPOSANT SECURITY

- Pour installer le composant Security dans un projet Symfony :

```
composer require symfony/security-bundle
```

CONFIGURATION DE LA SÉCURITÉ

- Le fichier `config/packages/security.yaml` permet de configurer la sécurité :

```yaml
security:
    encoders:
        App\Entity\Utilisateur:
            algorithm: auto
    providers:
        users_in_database:
            entity:
                class: App\Entity\Utilisateur
                property: email
```

CRÉATION D'UNE ENTITÉ UTILISATEUR

- Utilisation de la commande :

```
php bin/console make:user
```

CRÉATION D'UNE ENTITÉ UTILISATEUR

- Exemple d

```php
namespace App\Entity;
use Symfony\Component\Security\Core\User\UserInterface;
use Doctrine\ORM\Mapping as ORM;

#[ORM\Entity]
class Utilisateur implements UserInterface {
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column(type:'integer')]
    private int $id;

    #[ORM\Column(type:'string', unique:true)]
    private string $email;
```

CRÉATION DU SYSTÈME D'AUTHENTIFICATION

- Utilisation de la commande :

```
php bin/console make:auth
```

- Symfony génère automatiquement une authentification basée sur des formulaires.

AJOUT DE LA CONNEXION ET DÉCONNEXION

```php
use Symfony\Component\Security\Http\Authentication\AuthenticationUtils;
class SecurityController {
    #[Route('/login', name: 'app_login')]
    public function login(AuthenticationUtils $authenticationUtils) {
        $error = $authenticationUtils->getLastAuthenticationError();
        return $this->render('security/login.html.twig', ['error' => $error]);
    }
}
```

GESTION DES RÔLES ET PERMISSIONS

- - Symfony utilise le système de rôles pour restreindre l'accès :
- @IsGranted('ROLE_ADMIN')

SÉCURISATION DES ROUTES

- Exemple de sécurisation dans `security.yaml` :

- access_control:

  - { path: ^/admin, roles: ROLE_ADMIN }

PROTECTION CONTRE LES ATTAQUES CSRF

- Symfony intègre une protection CSRF automatique pour les formulaires.
- Ajout d'un token CSRF dans un formulaire :

```
$builder->add('_token', HiddenType::class, [
    'mapped' => false,
    'attr' => ['value' => csrf_token('auth')]
]);
```

RÉSUMÉ DE LA SÉCURITÉ DANS SYMFONY

- Gestion des utilisateurs avec `make:user`.
- Système d'authentification avec `make:auth`.
- Gestion des rôles et sécurisation des routes.
- Protection contre les attaques XSS et CSRF.

TESTS & DÉPLOIEMENT

FRAMEWORK SYMFONY

TESTS ET DÉPLOIEMENT DANS SYMFONY

Symfony offre des outils intégrés pour tester et déployer une application de manière efficace et sécurisée.

PRÉSENTATION DES TESTS DANS SYMFONY

- Symfony supporte les tests unitaires et fonctionnels avec PHPUnit.
- Permet de valider le bon fonctionnement du code avant le déploiement.

INSTALLATION DE PHPUNIT

- Pour installer PHPUnit dans un projet Symfony :

```
composer require --dev phpunit/phpunit
```

CRÉER UN TEST UNITAIRE

- Un test unitaire permet de tester une seule fonction ou méthode.
- Exemple :

```php
use PHPUnit\Framework\TestCase;
class CalculTest extends TestCase {
    public function testAddition() {
        $this->assertEquals(4, 2+2);
    }
}
```

CRÉER UN TEST FONCTIONNEL

- Les tests fonctionnels permettent de tester l'ensemble du fonctionnement d'une route ou d'un contrôleur.
- Exemple :

```php
use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;
class AccueilTest extends WebTestCase {
    public function testPageAccueil() {
        $client = static::createClient();
        $crawler = $client->request('GET', '/');
        $this->assertResponseIsSuccessful();
    }
}
```

EXÉCUTER LES TESTS

- Pour lancer les tests PHPUnit :

```
php bin/phpunit
```

DÉPLOIEMENT D'UNE APPLICATION SYMFONY

Le déploiement consiste à mettre l'application en production sur un serveur.

PRÉPARATION AU DÉPLOIEMENT

- Vérifier la configuration de l'environnement (`.env`).
- Exécuter les migrations :

```
php bin/console doctrine:migrations:migrate
```

UTILISATION DE SYMFONY CLI

- Symfony CLI permet de déployer facilement une application :

```
symfony server:deploy
```

COURS RÉALISÉ PAR FLORIAN MANCIERI